

Manual de Usuario

Sistema de Preguntas y Respuestas con RAG y Modelos LLM Locales

Cómo usar el sistema:

1. Ejecutar la aplicación

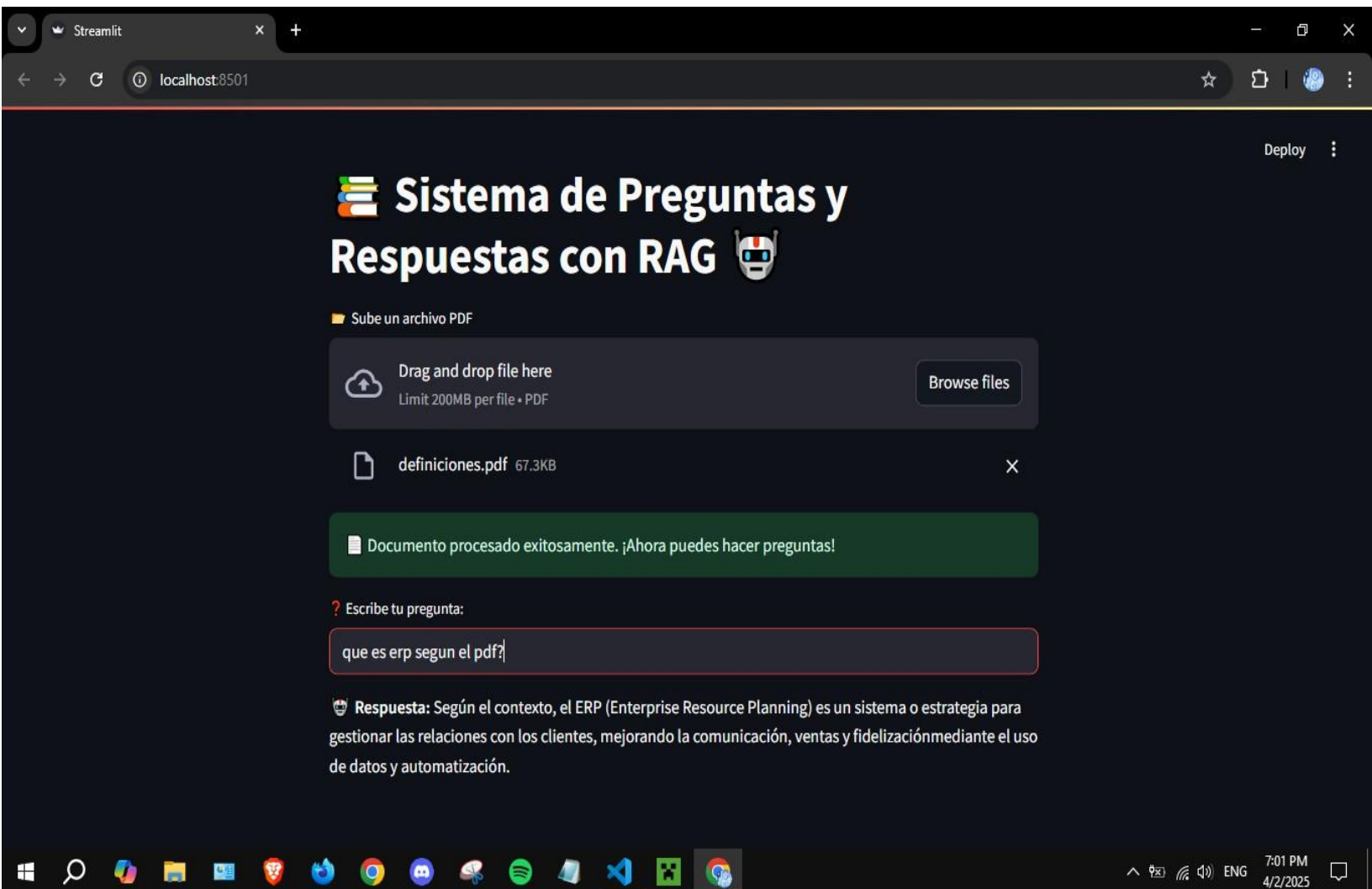
Desde terminal: `streamlit run app.py`

2. Subir un documento PDF

Haz clic en **"Examinar archivos"**.
Selecciona tu archivo PDF.
Espera a que se procese el texto del documento.

3. Realizar preguntas

Escribe tu pregunta en el campo de texto.
Presiona el botón **"Enviar"** o usa Enter.
La IA te responderá con base en el contenido del PDF ca



Manual Técnico

Tecnologías utilizadas

Python 3.10+

Streamlit – interfaz web

Ollama – ejecución de modelos LLM locales (gemma:2b)

LangChain – orquestación RAG

FAISS – vector store para recuperación semántica

PyPDF2 – extracción de texto de PDFs

Instalación de dependencias:

```
pip install streamlit langchain langchain_community langchain_core langchain_ollama  
pypdf pdfplumber pymupdf
```

link de tarea: https://github.com/TAMPADFF/chat_inteligente_python

ARCHIVO app.py

```
import streamlit as st  
from pdf_processor import extract_text_from_pdf  
from rag_model import create_vectorstore, answer_question  
  
st.title("📖 Sistema de Preguntas y Respuestas con RAG 🤖")  
  
# Subida de archivos PDF  
uploaded_file = st.file_uploader("📁 Sube un archivo PDF", type="pdf")  
  
if uploaded_file is not None:  
    with open(f"docs/{uploaded_file.name}", "wb") as f:  
        f.write(uploaded_file.getbuffer())  
  
    # Extraer texto del PDF  
    text = extract_text_from_pdf(f"docs/{uploaded_file.name}")  
  
    # Crear el vectorstore con embeddings  
    vectorstore = create_vectorstore(text)  
  
    st.success("📄 Documento procesado exitosamente. ¡Ahora puedes hacer preguntas!")  
  
    # Cuadro de entrada para preguntas del usuario  
    question = st.text_input("❓ Escribe tu pregunta:")
```

```

if question:
    response = answer_question(vectorstore, question)
    st.write("🤖 **Respuesta:**", response)

```

ARCHIVO pdf_processor.py

```

import pdfplumber

def extract_text_from_pdf(pdf_path):
    """Extrae el texto de un archivo PDF usando pdfplumber"""
    text = ""
    with pdfplumber.open(pdf_path) as pdf:
        for page in pdf.pages:
            text += page.extract_text() + "\n"
    return text

```

ARCHIVO rag_model.py

```

from langchain_community.llms import Ollama
from langchain_core.documents import Document
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import OllamaEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Cargar el modelo LLM de Ollama con Gemma:2b
llm = Ollama(model="gemma:2b")

def generate_embeddings(text_chunks):
    """Genera embeddings para los fragmentos de texto usando OllamaEmbeddings con Gemma."""
    embeddings = OllamaEmbeddings(model="gemma:2b")
    return FAISS.from_texts(text_chunks, embeddings)

def create_vectorstore(text):
    """Divide el texto en fragmentos y lo almacena en un vectorstore"""
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
    text_chunks = splitter.split_text(text)
    return generate_embeddings(text_chunks)

def answer_question(vectorstore, question):
    """Busca en la base de conocimiento y responde preguntas usando RAG"""
    docs = vectorstore.similarity_search(question, k=3) # Recuperar los 3 documentos más relevantes
    context = "\n".join([doc.page_content for doc in docs])

```

```
prompt = f"""
Contexto: {context}
Pregunta: {question}
Responde de manera precisa basándote en el contexto.
"""

return llm.invoke(prompt)
```